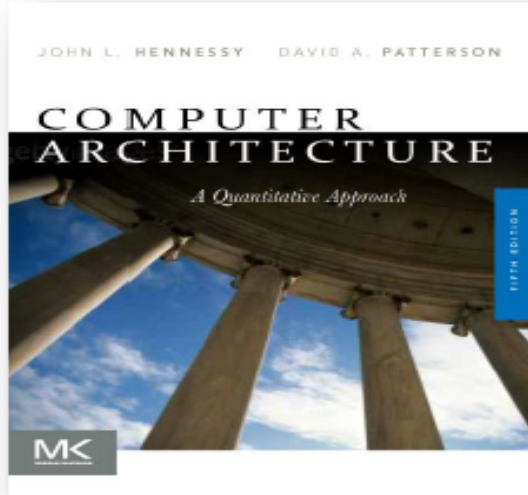
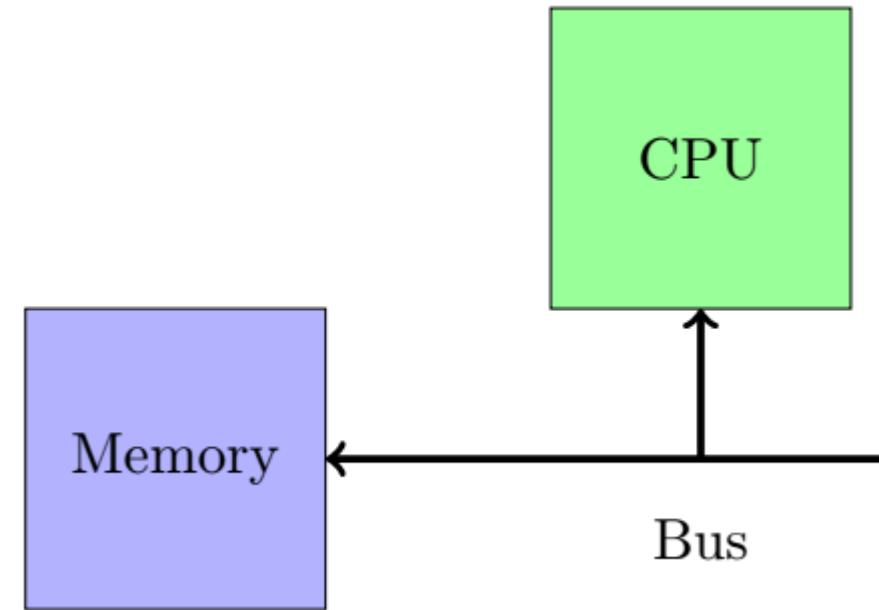
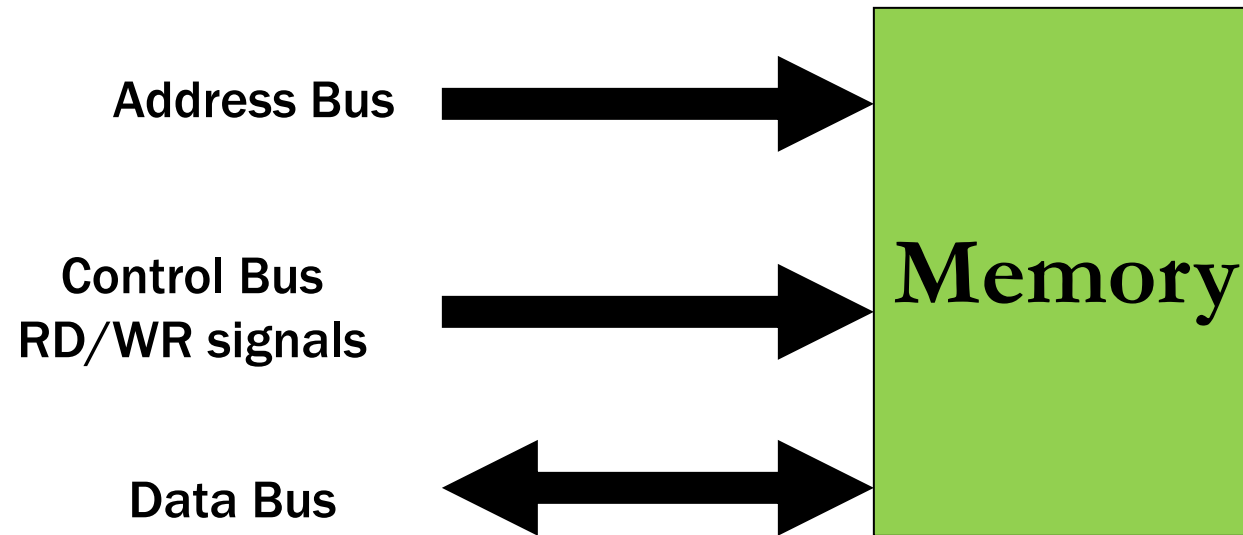




Chapter 2

Memory Hierarchy Design

Accessing Memory



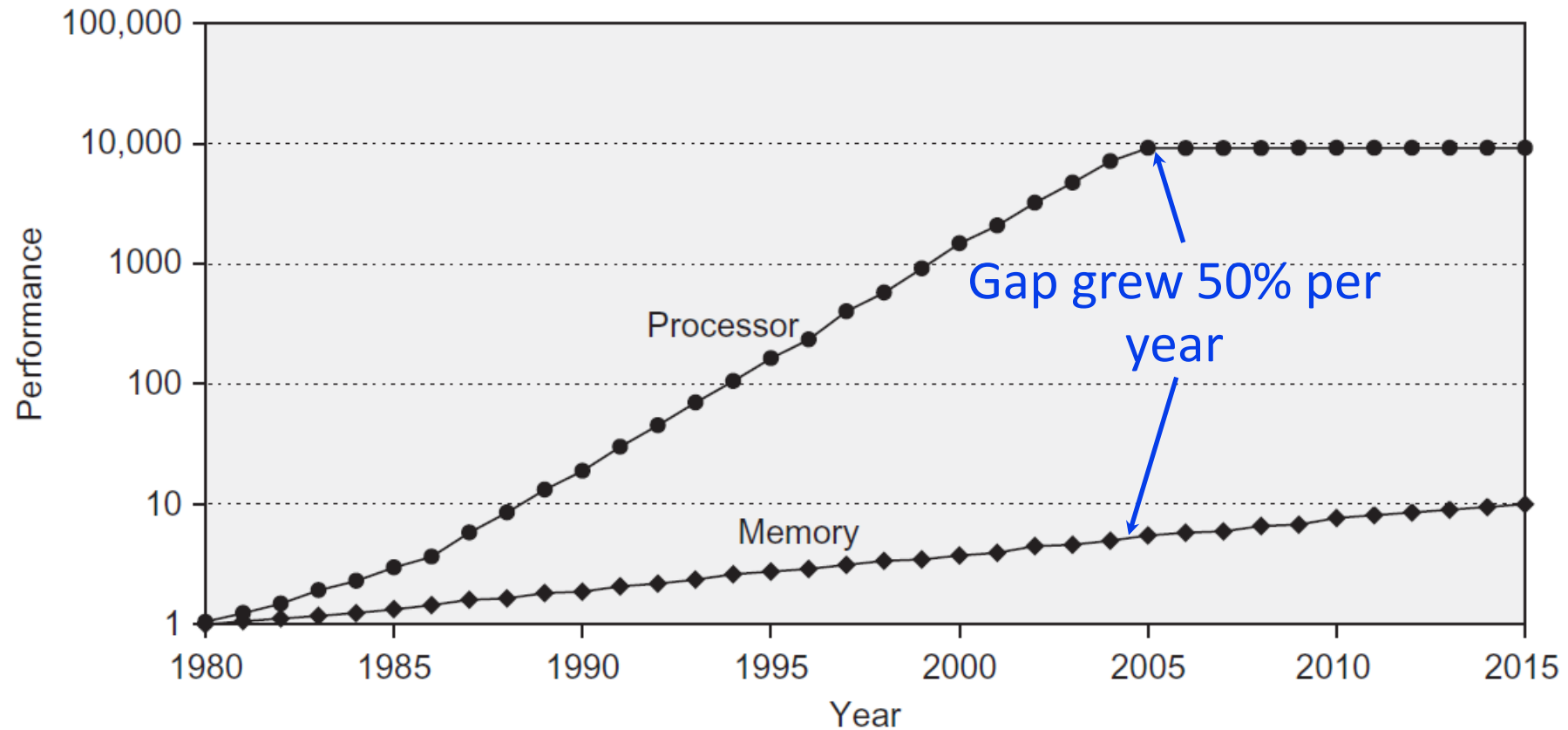
Reading From & Writing to Memory

- Reading from memory
 - **Address** of memory location to read is placed on Address Bus
 - **Read Signal** (RD) in control bus is activated
 - **Data** is fetched (read) from Data Bus
- Writing to memory
 - **Address** of memory location to write is placed on Address Bus
 - **Data** is placed on Data Bus
 - **Write Signal** (WR) in control bus is activated

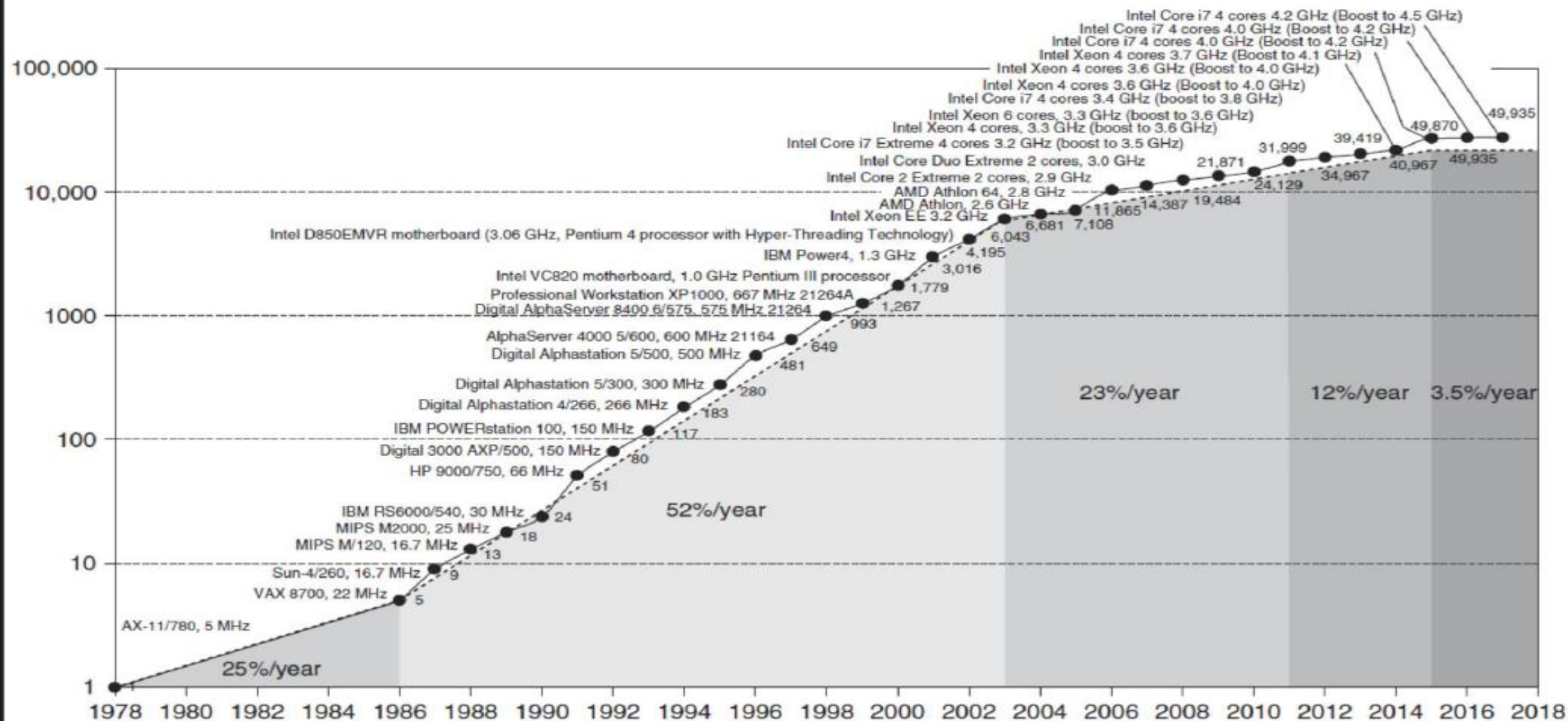
Introduction

- Programmers want unlimited amounts of memory with low latency
- Fast memory technology is more expensive per bit than slower memory
- Solution: organize memory system into a hierarchy
 - Entire addressable memory space available in largest, slowest memory
 - Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor
- Temporal and spatial locality insures that nearly all references can be found in smaller memories
 - Gives the illusion of a large, fast memory being presented to the processor

Processor Memory Gap



Single Processor Performance



Memory Hierarchy Design

- Memory hierarchy design becomes more crucial with recent multi-core processors:
 - Aggregate peak bandwidth grows with # cores:
 - Intel Core i7 can generate two references per core per clock
 - Four cores and 3.2 GHz clock
 - 25.6 billion 64-bit data references/second +
 - 12.8 billion 128-bit instruction references/second
 - = 409.6 GB/s!
 - DRAM bandwidth is only 8% of this (34.1 GB/s)
 - Requires:
 - Multi-port, pipelined caches
 - Two levels of cache per core
 - Shared third-level cache on chip

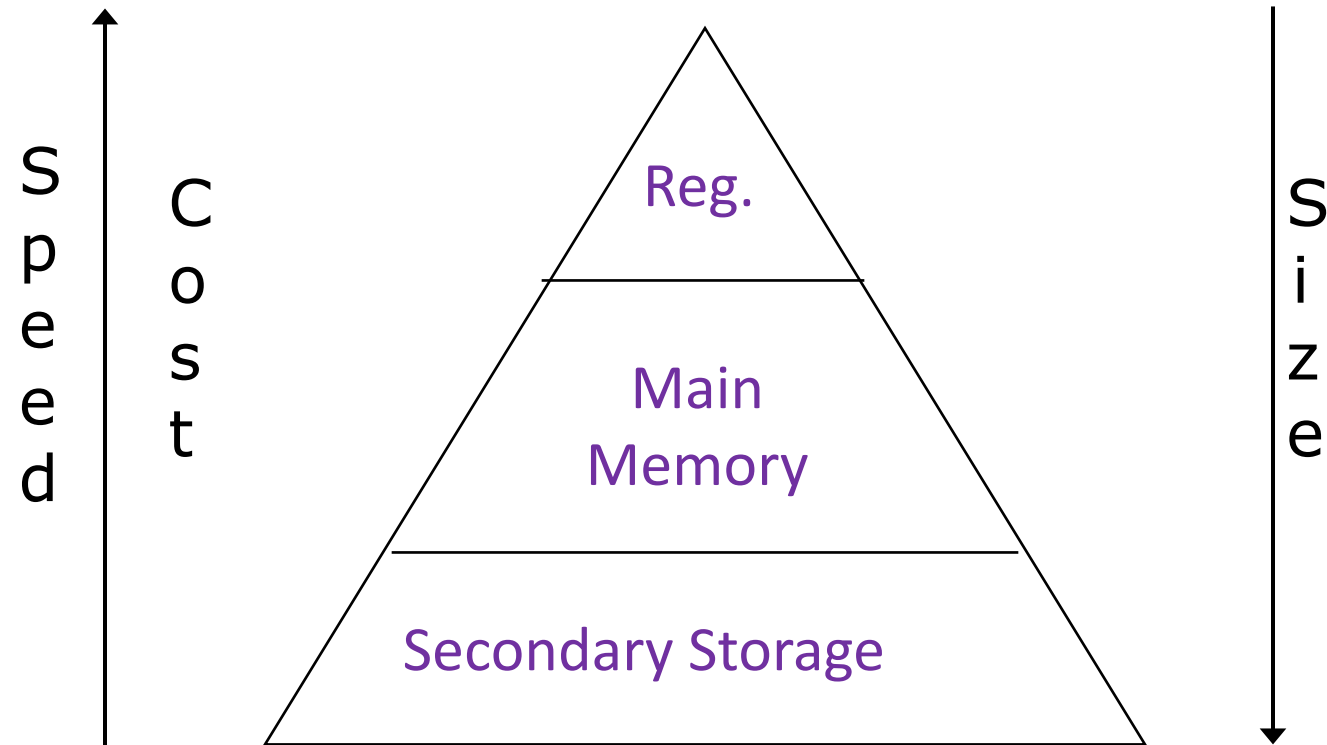
Memory Hierarchy (Cont.)

- Objective is to have a memory system
 - with sufficient speed
 - with sufficient capacity
 - as cheap as possible

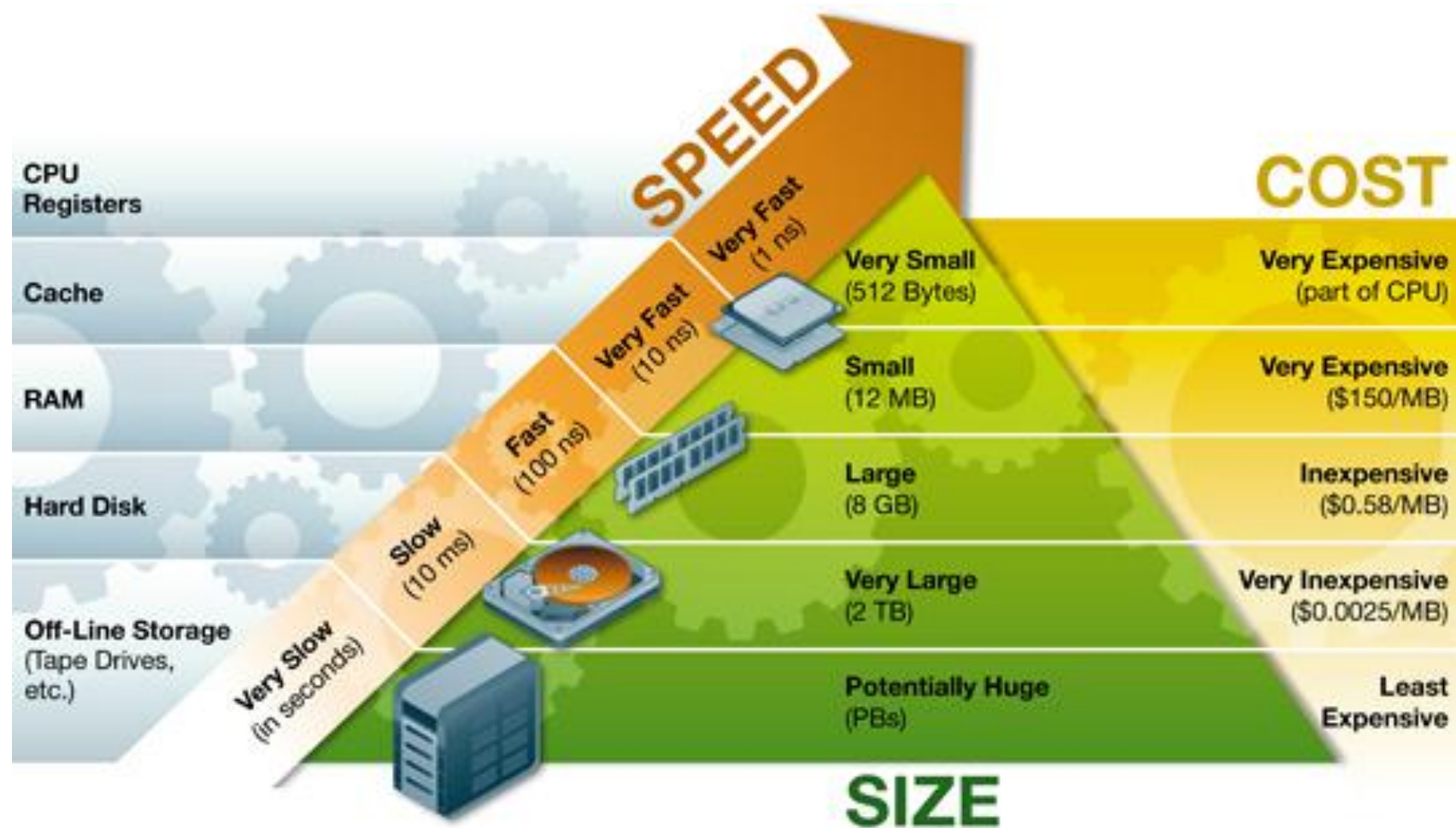
Ideally one would desire an indefinitely large memory capacity such that any particular... word would be immediately available... We are... forced to recognize the possibility of constructing a hierarchy of memories each of which has greater capacity than the preceding but which is less quickly accessible.

A. W. Burks, H. H. Goldstine,
and J. von Neumann,
*Preliminary Discussion of the
Logical Design of an Electronic
Computing Instrument* (1946).

Traditional Memory Hierarchy

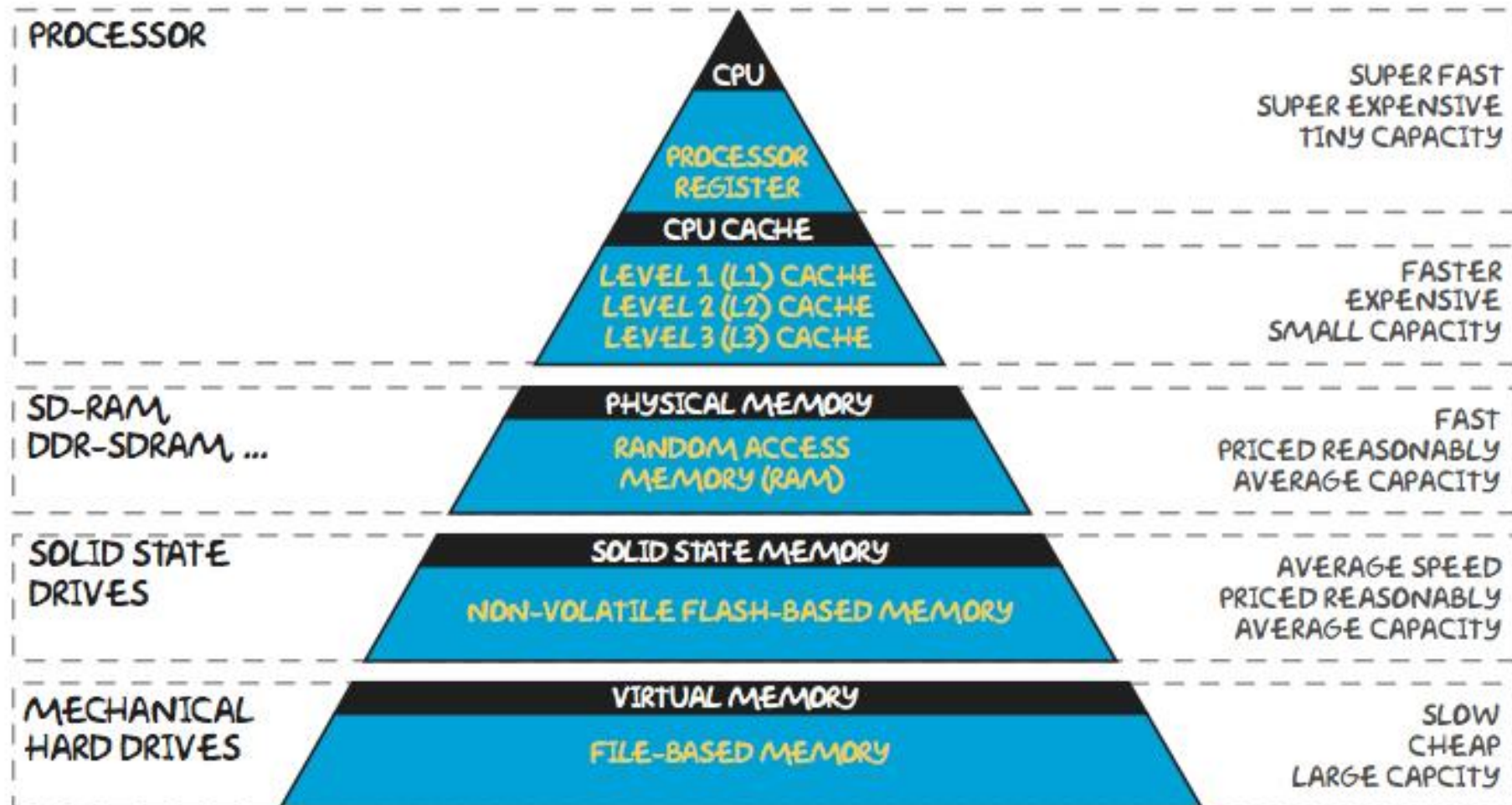


Extended Memory Hierarchy



Source: http://www.ts.avnet.com/uk/products_and_solutions/storage/hierarchy.html

Modern Memory Hierarchy



Principle of Locality

- Programs tend to reuse data & instructions that are close to each other or they have used recently
- Temporal locality
 - Recently referenced items are likely to be referenced in the near future
 - A block tend to be accessed again & again
- Spatial locality
 - Items with nearby addresses tend to be referenced close together in time
 - Near by blocks tend to be accessed

Locality – Example

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```



- Data
 - Access array elements in succession – Spatial locality
 - Reference `sum` each iteration – Temporal locality
- Instructions
 - Reference instructions in sequence – Spatial locality
 - Cycle through loop repeatedly – Temporal locality

Locality examples

- Loops
- Sequential instructions
- Array elements

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Prefetching – Example

- Which of the following code is faster?

```
sum = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        sum += a[i][j];
return sum;
```

```
sum = 0;
for (j = 0; j < m; j++)
    for (i = 0; i < n; i++)
        sum += a[i][j];
return sum;
```

Programmer's view of matrix

	Col 1	Col 2	Col 3	Col 4
Row 1	1, 1	1, 2	1, 3	1, 4
Row 2	2, 1	2, 2	2, 3	2, 4
Row 3	3, 1	3, 2	3, 3	3, 4

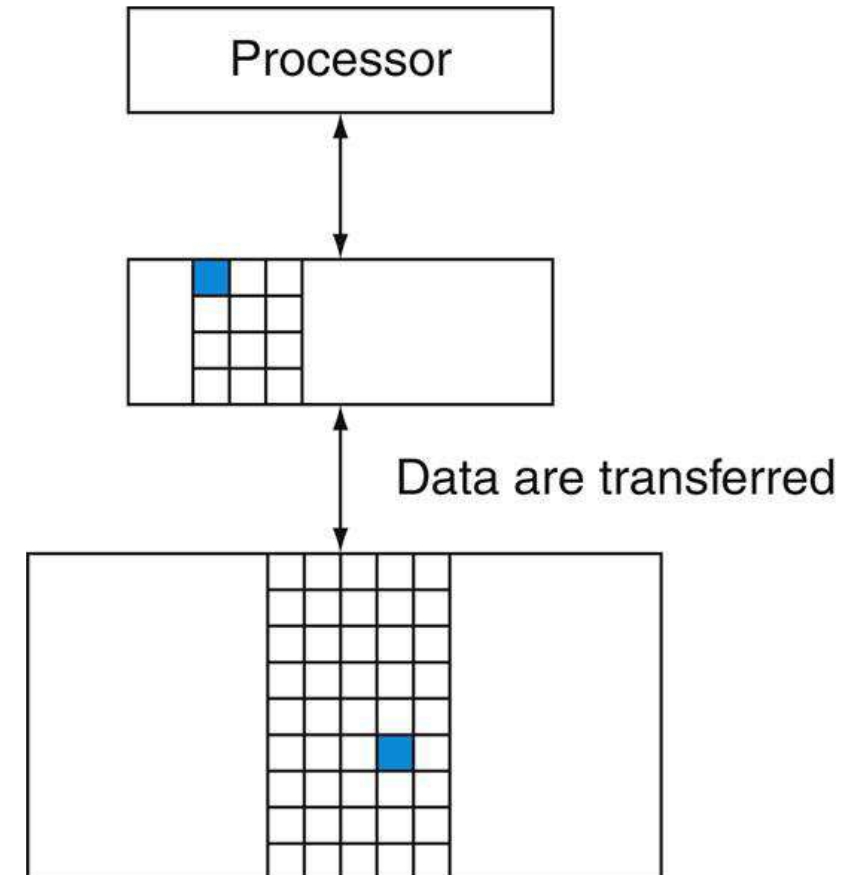
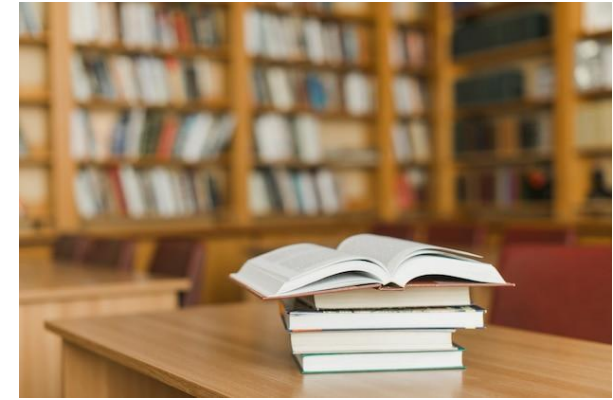
1,1	1,2	1,3	1,4	2,1	2,2	2,3	2,4	3,1	...		
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--	--

Impact of Hierarchical Memory

- Memory is no more a flat structure
- Affect many aspects of a computer
 - How OS manages memory & IO
 - How compilers generate codes
 - How applications use the computers
- Programs spend more time on memory access
 - Processors *stall* waiting for data from memory
- Need to know hierarchy for optimizing

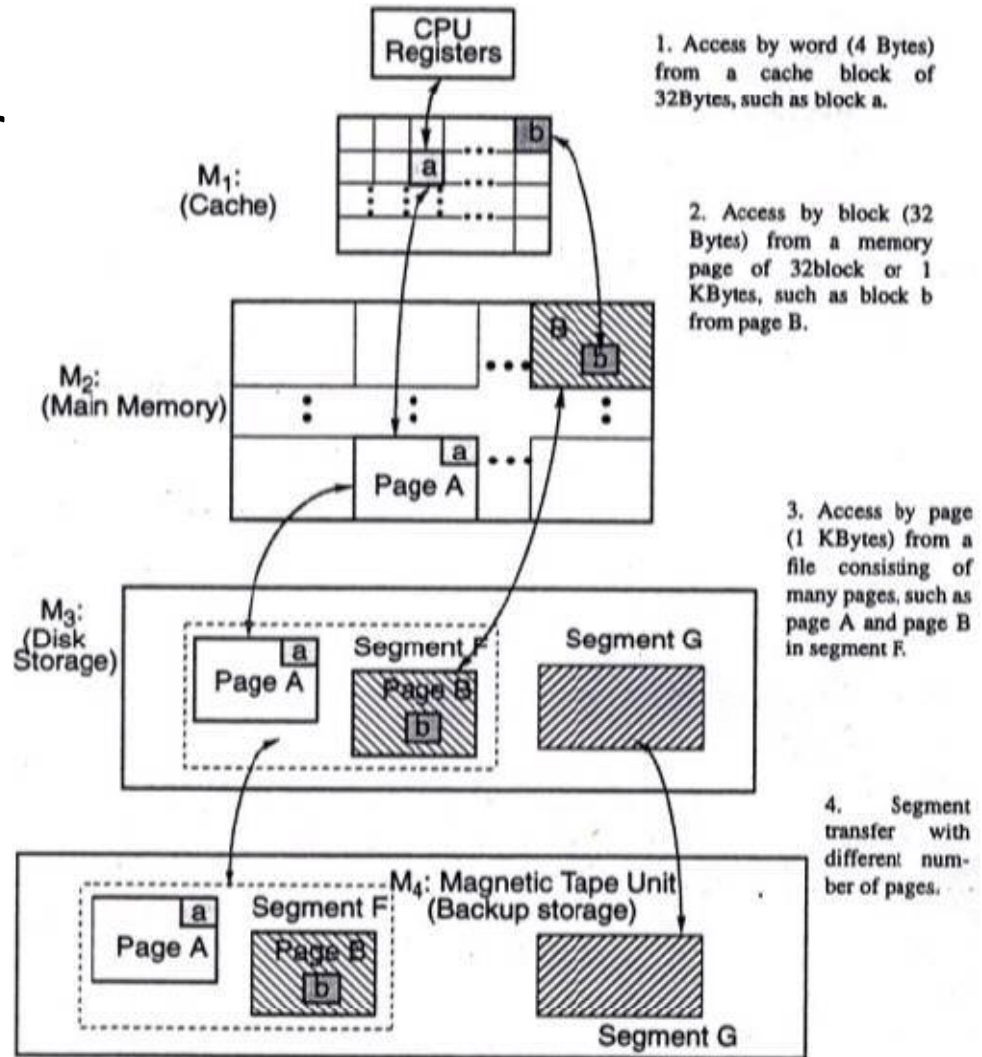
Cache

- Small amount of memory that is faster than DRAM
 - Slower than registers
 - Built using SRAM
 - Range from few KB to few MB
- Used by CPU to store frequently used instructions & data
 - Spatial & temporal locality
- Use multiple levels of cache
 - L1 Cache – Very fast, usually within CPU itself
 - L2 Cache – Slower than L1, but faster than DRAM
 - Today there's even L3 Cache



Inclusion Property

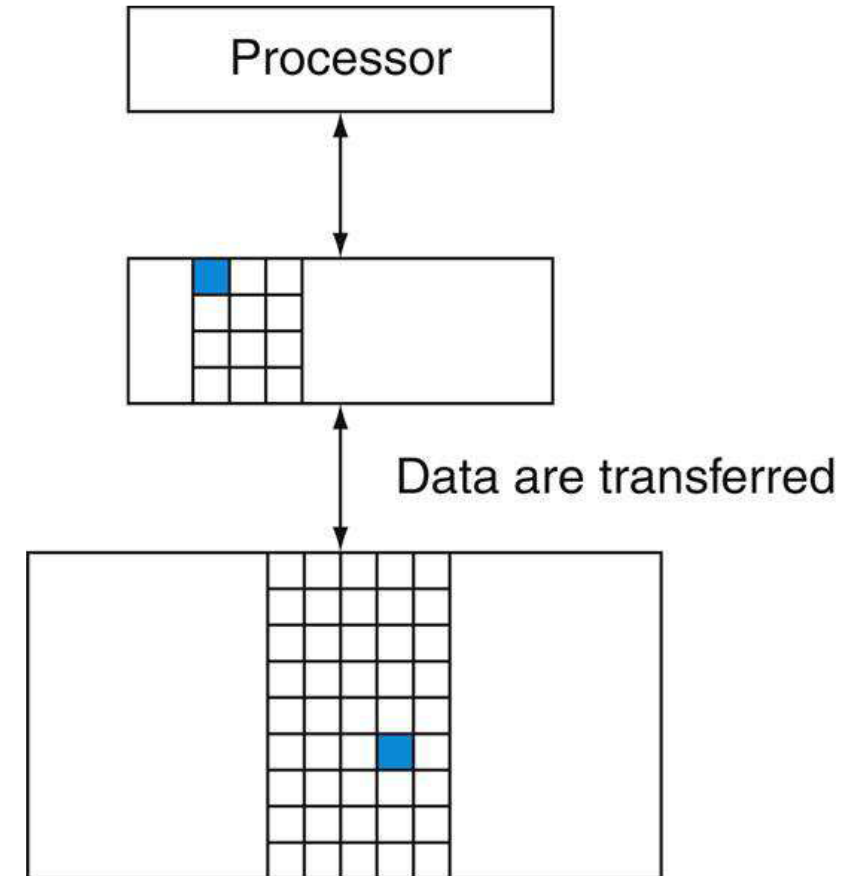
- Often data in lower levels of hierarchy are a **superset** of higher level memories
- Terminology
 - Block (=line)
 - Hit, Miss
 - Hit rate, Miss rate
 - Hit time, Miss penalty



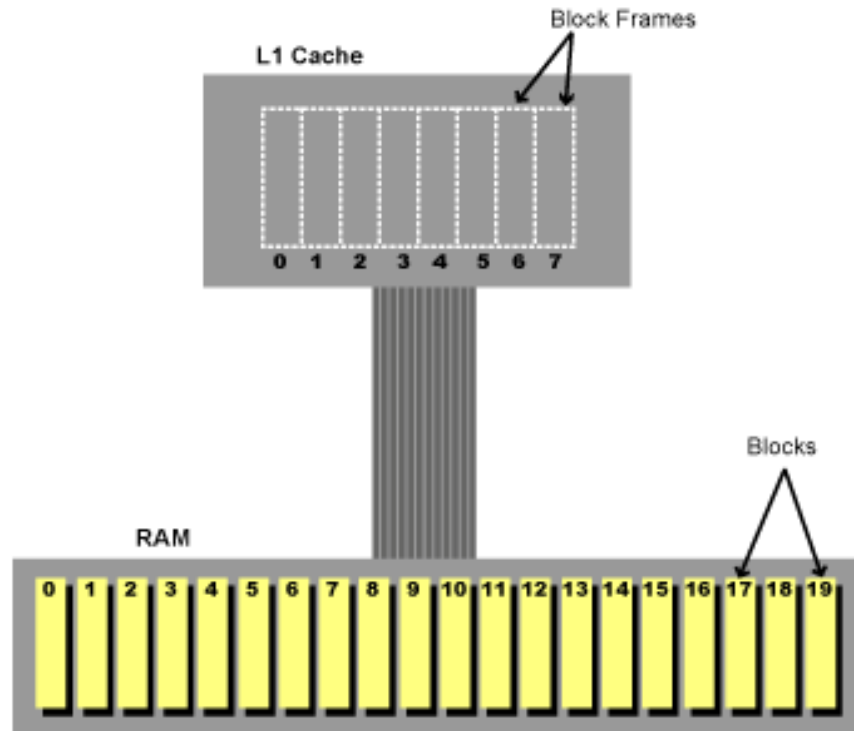
The inclusion property and data transfers between adjacent levels

Cache - terminology

- Minimum unit of data: **block = line**
- If data requested by the processor
 - is found in Cache: **hit**
 - Is NOT found in Cache: **miss**
- **Hit ratio**: fraction of memory accesses found in cache
- **Miss ratio**: fraction of memory accesses not found in cache
- **Hit time**: time to access a block in cache
- **Miss penalty**: time to bring a block from memory plus deliver it to processor



Cache Blocks



Source:

<http://archive.arstechnica.com/paedia/c/caching/m-caching-5.html>

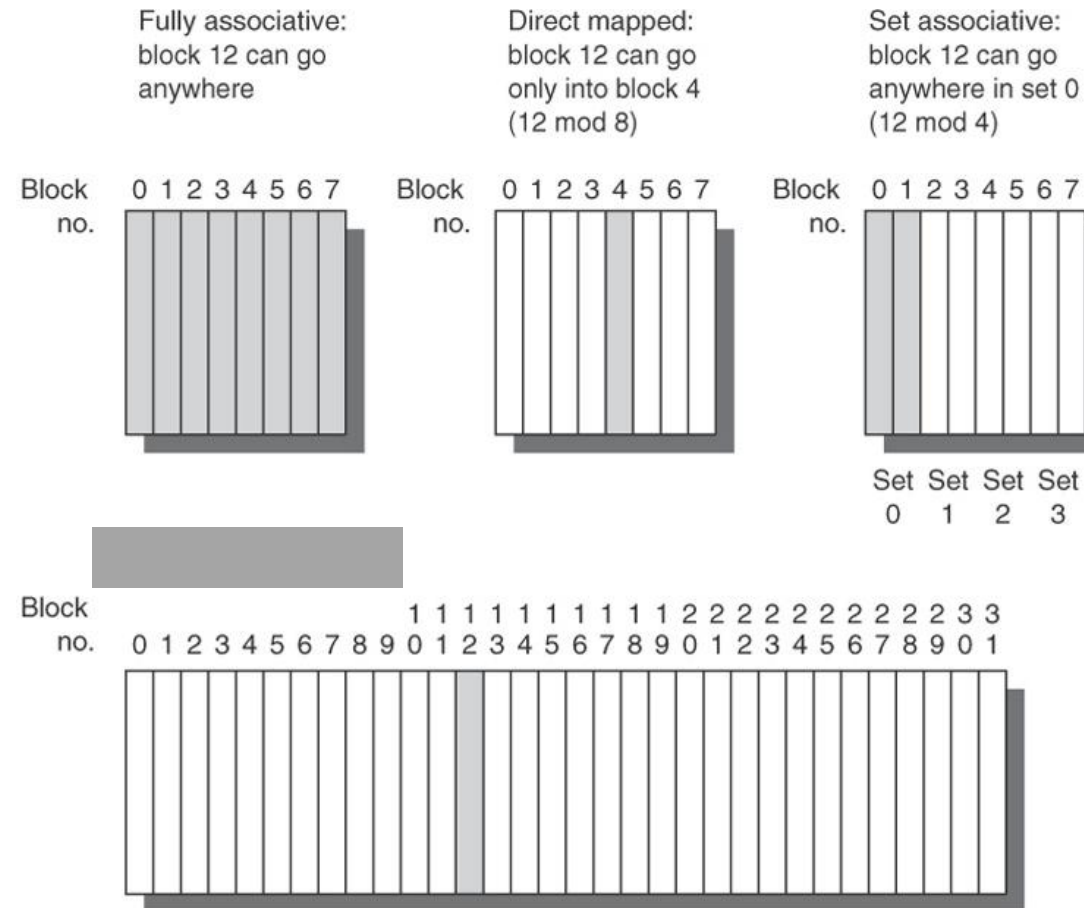
- A collection of *words* are called a *block*
- Multiple blocks are moved between levels in cache hierarchy
- Blocks are tagged with memory address
 - Tags are searched parallel

Memory Hierarchy Basics

- When a word is not found in the cache, a *miss* occurs:
 - Fetch word from lower level in hierarchy, requiring a higher latency reference
 - Lower level may be another cache or the main memory
 - Also fetch the other words contained within the *block*
 - Takes advantage of spatial locality
 - Place block into cache in any location within its *set*, determined by address
 - $\text{block address MOD number of sets in cache}$

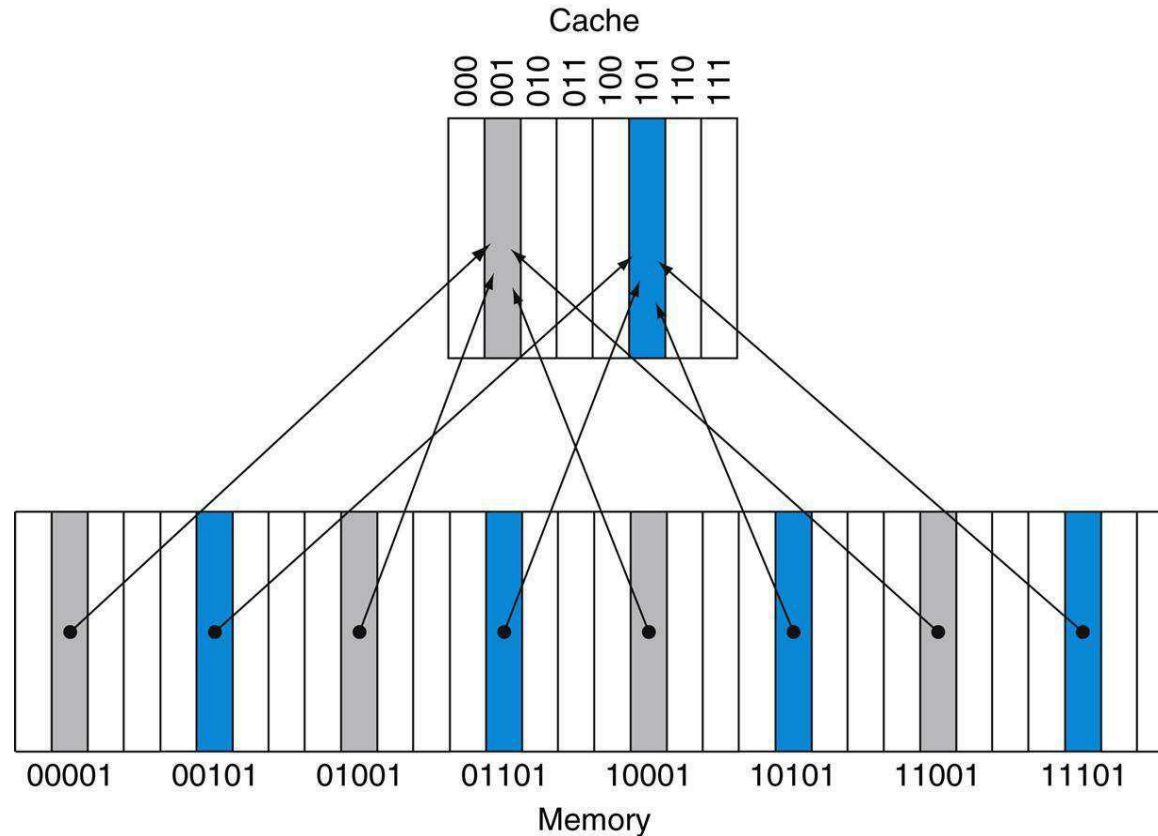
Cache Associativity

- Defines where blocks can be placed in a cache



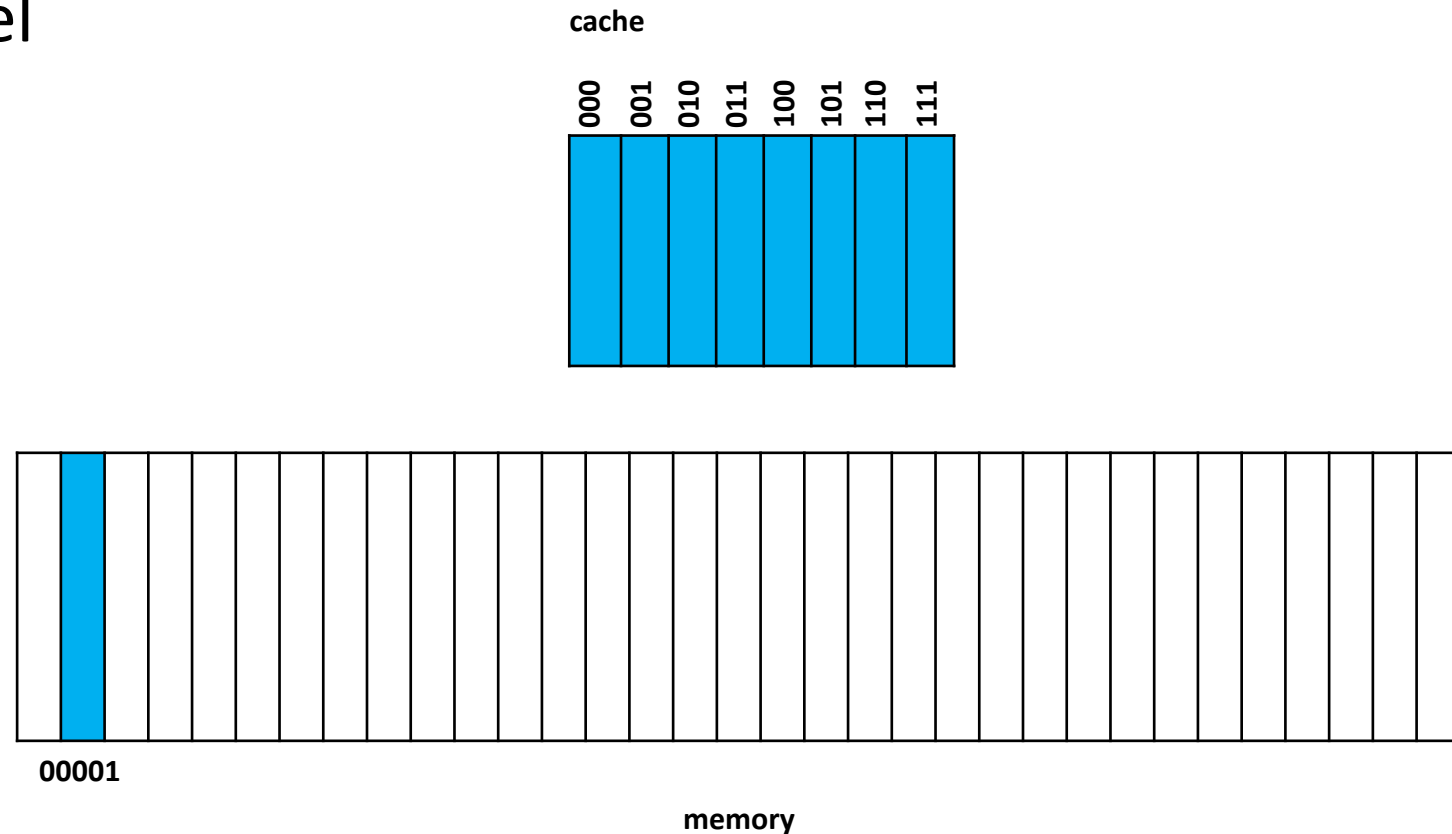
Direct mapping

- A memory block can go exactly to one place in cache



Fully associative

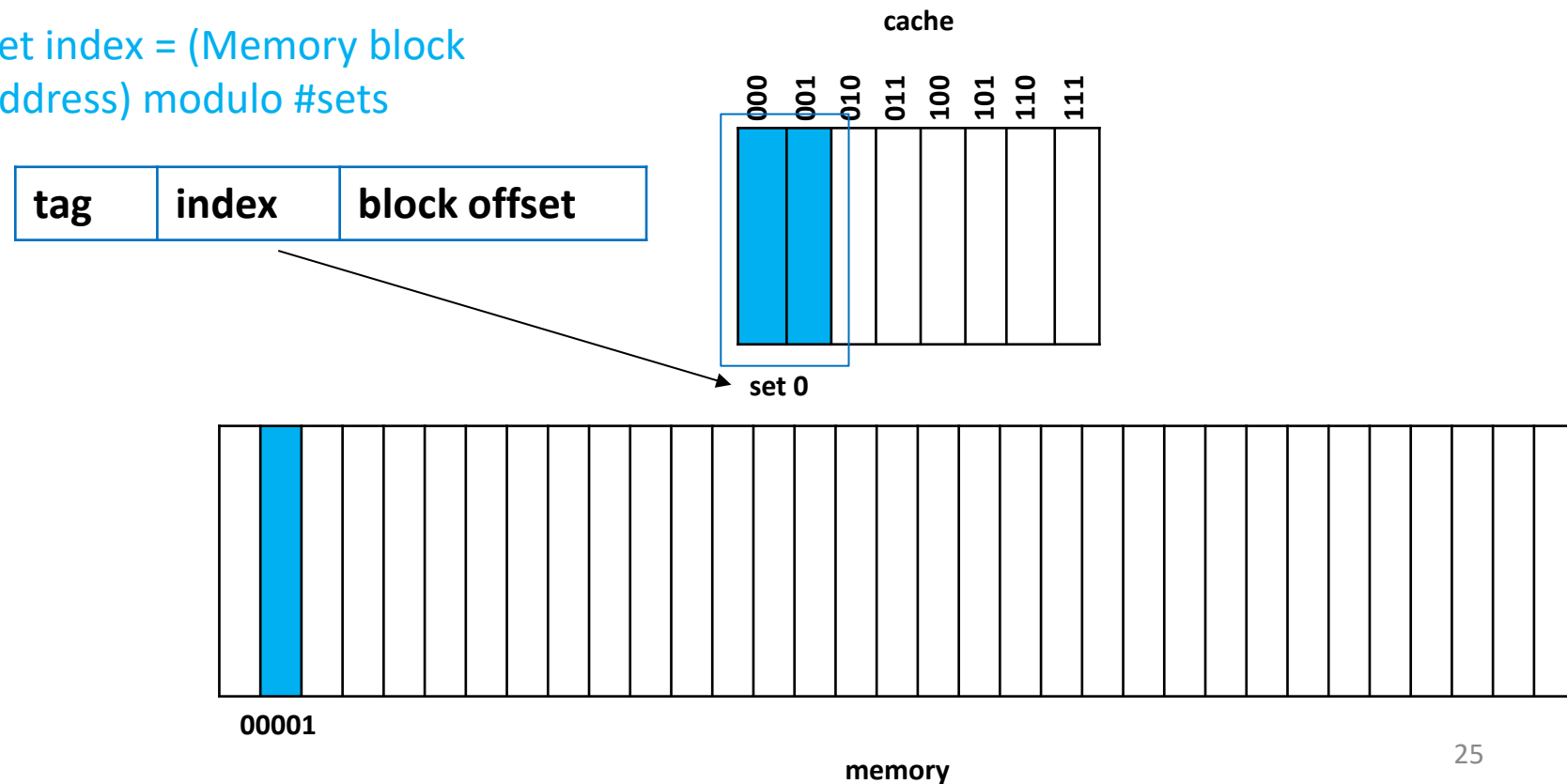
- A block in main memory can go to any block in cache
- Searched parallel



Set associative

- A block in main memory can go to any block in a set of cache blocks
- *n-way set associative*: n blocks for a set

Set index = (Memory block address) modulo #sets



Cache configuration examples

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

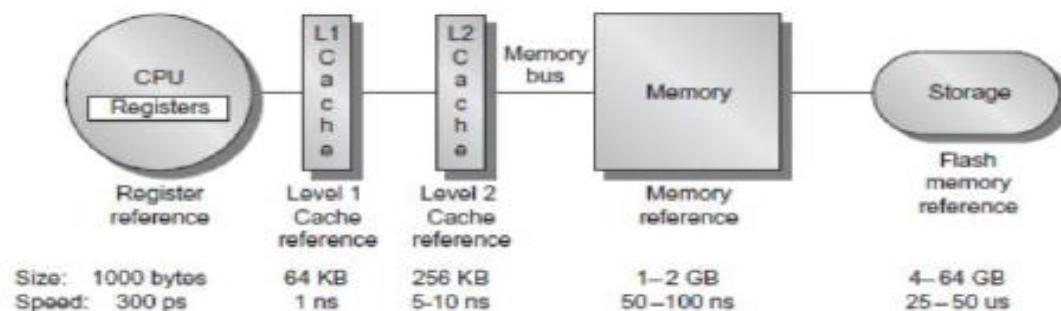
Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

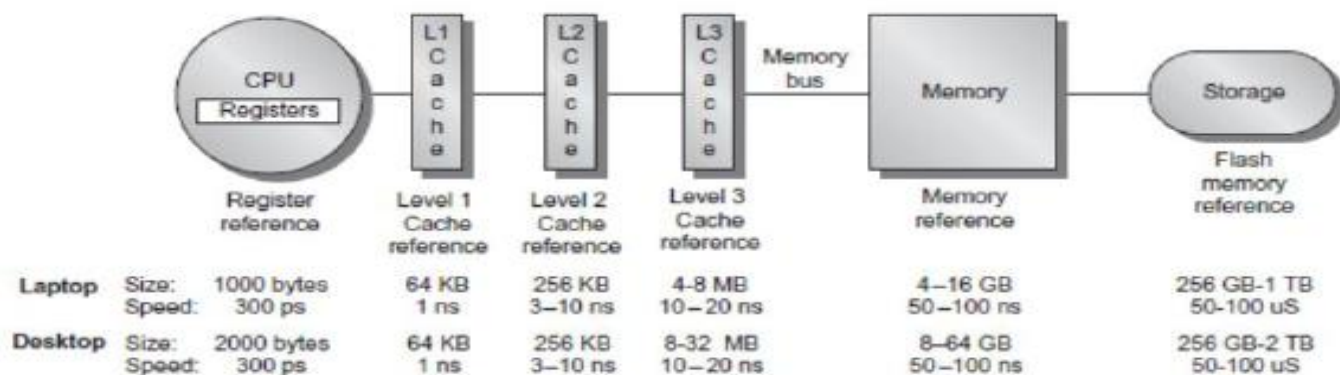
Memory Hierarchy Basics

- n sets $\Rightarrow$ n -way set associative
 - *Direct-mapped cache* $\Rightarrow$ one block per set
 - *Fully associative* $\Rightarrow$ one set
- Writing to cache: two strategies
 - *Write-through*
 - Immediately update lower levels of hierarchy
 - *Write-back*
 - Only update lower levels of hierarchy when an updated block is replaced
 - Both strategies use *write buffer* to make writes asynchronous

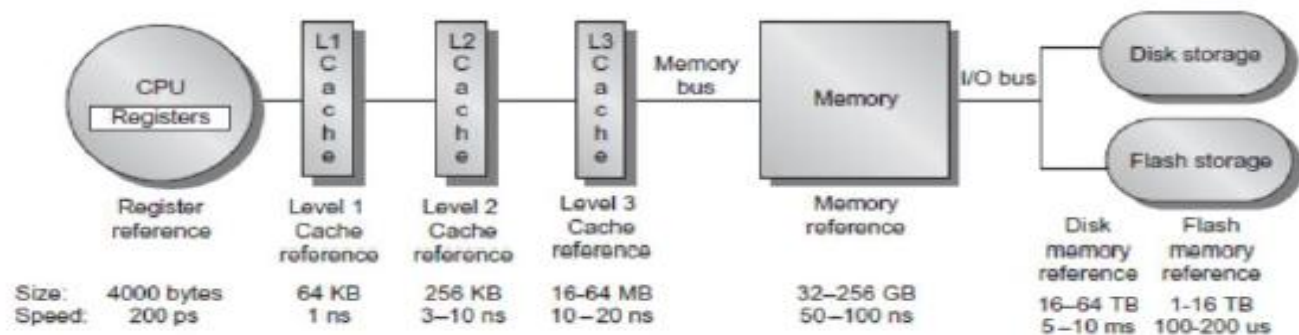
Memory Hierarchy



(A) Memory hierarchy for a personal mobile device



(B) Memory hierarchy for a laptop or a desktop



(C) Memory hierarchy for server

Memory Hierarchy Basics

- Miss rate
 - Fraction of cache access that result in a miss
- Causes of misses
 - Compulsory
 - First reference to a block
 - Capacity
 - Blocks discarded and later retrieved
 - Conflict
 - Program makes repeated references to multiple addresses from different blocks that map to the same location in the cache

Memory Hierarchy Basics

$$\frac{\text{Misses}}{\text{Instruction}} = \frac{\text{Missrate} \times \text{Memory accesses}}{\text{Instruction count}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

- Speculative and multithreaded processors may execute other instructions during a miss
 - Reduces performance impact of misses

Memory Hierarchy Basics

- Six basic cache optimizations:
 - Larger block size
 - Reduces compulsory misses
 - Increases capacity and conflict misses, increases miss penalty
 - Larger total cache capacity to reduce miss rate
 - Increases hit time, increases power consumption
 - Higher associativity
 - Reduces conflict misses
 - Increases hit time, increases power consumption
 - Higher number of cache levels
 - Reduces overall memory access time
 - Giving priority to read misses over writes
 - Reduces miss penalty
 - Avoiding address translation in cache indexing
 - Reduces hit time

Thank you